Technical Documentation: Road Quality and Driver Metrics Analysis

This document explains the physics, formulas, and algorithms used in **Tracker** to compute road quality metrics, detect road features, and evaluate driver behavior using smartphone accelerometer and GPS data.

Table of Contents

1. Sensor Data Acquisition
2. Coordinate Systems and Transformations
3. Accelerometer Data Processing
4. Road Quality Classification
5. Road Feature Detection
6. Driver Event Detection
7. Driver Smoothness Scoring
8. Lean Angle Calculation
9. Vehicle Profiles
10. Threshold Calibration from Recorded Tracks
11. KML Output and Visualisation

Sensor Data Acquisition

GPS Data

- **Sampling Rate**: Configurable (default 1 Hz)
- **Data Points**: Latitude, longitude, altitude, speed, bearing, accuracy, timestamp
- **Coordinate System**: WGS84 geographic coordinates
- **Speed Calculation**: GPS-provided speed in m/s, converted to km/h for display

Accelerometer Data

- **Sampling Rate**: Typically 50-100 Hz (device-dependent)
- **Data Points**: X, Y, Z acceleration values in m/s^2
- **Units**: m/s^2 (including gravity)
- **Coordinate System**: Device frame (varies with phone orientation)

Coordinate Systems and Transformations

Device Frame to Vehicle Frame

The accelerometer data is transformed from the device coordinate system to the vehicle frame to ensure consistent analysis regardless of phone mounting orientation.

Gravity Vector Removal

First, the gravity component is removed from raw accelerometer readings:

```
a_linear = a_raw - g_vector
```

Where: - `a_raw` = Raw accelerometer reading [x, y, z] - `g_vector` = Gravity vector (captured during baseline calibration) - `a_linear` = Linear acceleration (vehicle motion only)

Vehicle Frame Projection

The linear acceleration is projected onto vehicle axes:

```
a_fwd = a_linear · unit_vector_forward
a_lat = a_linear · unit_vector_lateral
a_vert = a_linear · unit_vector_vertical
```

Where the unit vectors are derived from the gravity vector to align with: - Forward: Direction of vehicle travel - Lateral: Perpendicular to forward (left-right) - Vertical: Up-down relative to vehicle

Screen Rotation Compensation

The accelerometer axes are remapped based on screen rotation:

```
if (rotation == ROTATION_90):
    swap(x, y); x = -x
elif (rotation == ROTATION_270):
    swap(x, y); y = -y
```

Accelerometer Data Processing

Detrending

A moving average is used to remove low-frequency drift:

```
avg_window = 5 samples
detrended = a_linear - moving_average(a_linear, avg_window)
```

Statistical Metrics

For each GPS fix interval (typically 1 second), the following metrics are computed from the high-frequency accelerometer samples:

RMS (Root Mean Square)

```
rms = sqrt(mean(a_linear²))
```

Standard Deviation

```
std_dev = sqrt(variance(a_linear))
```

Peak Values

```
peak = max(|a_linear|)
```

Peak Ratio

```
peak_ratio = peak / rms
```

These metrics are computed separately for each axis (forward, lateral, vertical) and for the magnitude vector.

Road Quality Classification

Road quality is classified based on the statistical properties of vertical acceleration, which correlates with road surface roughness.

Classification Algorithm

For each GPS point, the road quality is determined by comparing RMS and standard deviation thresholds:

```
if (rms_z <= rms_smooth_max && std_dev_z <= std_dev_smooth_max):
    quality = "smooth"
elif (rms_z >= rms_rough_min && std_dev_z >= std_dev_rough_min):
    quality = "rough"
else:
    quality = "average"
```

Physics Basis

- **Smooth Roads**: Low vibration amplitude and variability
- **Rough Roads**: High vibration amplitude and variability
- **Average Roads**: Intermediate characteristics

Threshold Parameters

- `rms_smooth_max`: Maximum RMS for smooth classification (default: 1.0 m/s^2)
- `std_dev_smooth_max`: Maximum std dev for smooth classification (default: 2.5 m/s^2)

- `rms_rough_min`: Minimum RMS for rough classification (default: 4.5 m/s^2)
- `std_dev_rough_min`: Minimum std dev for rough classification (default: 3.0 m/s^2)

Road Feature Detection

Road features (bumps and potholes) are detected per GPS fix using the vertical acceleration statistics computed over that fix's accelerometer window.

Detection Algorithm

Feature detection runs after road quality metrics are computed for a fix. A fix is classified as a feature if two conditions are both met:

1. **Roughness gate**: The vertical RMS exceeds `rms_rough_min` — i.e. the fix is at least as rough as the rough threshold. This prevents false detections on smooth roads.
2. **Severity gate**: The maximum acceleration magnitude across all axes exceeds `mag_max_severe_min` — i.e. there is a sharp, high-energy impulse.

```
if rms_vert > rms_rough_min AND mag_max > mag_max_severe_min:
    if mean_vert < 0:
        feature = "bump"
    else:
        feature = "pothole"
else:
    feature = null
```

Bump vs Pothole Discrimination

Once a fix passes both gates, the **sign of the mean vertical acceleration** (`mean_vert`) over the fix window determines the feature type:

`mean_vert` sign	Physical interpretation	Classification
Negative (`< 0`)	Net downward impulse — vehicle drops into a depression	**Bump**
Positive (`≥ 0`)	Net upward impulse — vehicle rides over a raised obstacle	**Pothole**

This sign convention reflects the vehicle-frame vertical axis: gravity is removed, so a downward impulse (vehicle falling into a hole) produces a negative mean, while an upward impulse (wheel hitting a raised bump) produces a positive mean.

Feature Isolation

Fixes where a road feature is detected are **excluded from road quality history** and **excluded from driver event detection**. This prevents a bump or pothole from

contaminating the road quality moving average or triggering false hard-brake/swerve events.

Threshold Parameters

- `rms_rough_min`: Roughness gate (same parameter as road quality rough threshold)
- `mag_max_severe_min`: Severity gate — minimum peak magnitude to qualify as a feature (default: 20.0 m/s^2)
- `peak_threshold_z`: Used in the moving-average detrending step (default: 1.5 m/s^2)

Driver Event Detection

Driver events are detected from forward and lateral acceleration patterns.

Hard Braking

- **Physics**: Rapid deceleration causes negative forward acceleration
- **Detection**: `a_fwd < -hard_brake_fwd_max` (default: -15 m/s^2)
- **Conditions**: Speed > `min_speed_kmph` (default: 6 km/h)

Hard Acceleration

- **Physics**: Rapid acceleration causes positive forward acceleration
- **Detection**: `a_fwd > hard_accel_fwd_max` (default: 15 m/s^2)
- **Conditions**: Speed > minimum threshold

Swerving

- **Physics**: Lateral acceleration during sudden lane changes
- **Detection**: `|a_lat| > swerve_lat_max` (default: 4 m/s^2)
- **Conditions**: Speed > minimum threshold

Aggressive Cornering

- **Physics**: High lateral acceleration during turns
- **Detection**: `|a_lat| > aggressive_corner_lat_max` (default: 4 m/s^2)
- **Additional**: Course change rate > `aggressive_corner_dcourse` (default: 15°/s)

Course Change Calculation

```
course_change = abs(current_bearing - previous_bearing)
if course_change > 180:
    course_change = 360 - course_change
course_change_rate = course_change / time_interval
```

Driver Smoothness Scoring

The smoothness score evaluates driving quality based on acceleration variability.

Scoring Algorithm

For each GPS point above minimum speed:

```
smoothness_score = max(0, 100 - (rms_fwd * smoothness_rms_max_factor))
```

Where: - `smoothness_rms_max_factor` converts RMS to penalty points (default: 10) - Higher RMS acceleration = lower smoothness score - Score range: 0-100 (100 = perfectly smooth)

Overall Smoothness

The final smoothness score is the average of all point scores during the trip:

```
overall_smoothness = mean(all_smoothness_scores)
```

Lean Angle Calculation

Lean angle is computed from the lateral and vertical acceleration components.

Physics Principle

When a vehicle leans, the gravity vector shifts relative to the vehicle frame. The lean angle can be derived from the ratio of lateral to vertical acceleration.

Calculation Formula

```
lean_angle = atan2(a_lat, a_vert) * (180 / π)
```

Where: - `a_lat` = Lateral acceleration (m/s^2) - `a_vert` = Vertical acceleration including gravity (m/s^2) - Result in degrees (positive = right lean, negative = left lean)

Calibration Considerations

The calculation assumes the phone is mounted vertically. For horizontal mounting, the formula would use different axis combinations.

Vehicle Profiles

Thresholds are stored in named vehicle profiles (`.profile.json` files) in the save folder. Three default profiles are created on first launch:

Profile	RMS Smooth Max	StdDev Smooth Max	RMS Rough Min	StdDev Rough Min	MagMax Severe Min

Profile	RMS Smooth Max	StdDev Smooth Max	RMS Rough Min	StdDev Rough Min	MagMax Severe Min
Motorcycle	1.0	2.5	4.5	3.0	20.0
Car	1.5	3.0	5.0	3.5	20.0
Bicycle	0.8	2.0	3.5	2.5	15.0

Profiles also store all driver threshold settings. The active profile name is persisted in DataStore. Loading a profile writes all its values to the live DataStore so they take effect immediately without restarting recording.

Threshold Calibration from Recorded Tracks

Tracker includes an automated **Calibrate Thresholds** feature that analyses a previously recorded JSON track and recommends optimal road quality and driver threshold values for the specific vehicle and road conditions encountered during that recording.

Overview

The calibration engine (`ThresholdRecommendationEngine`) is a pure Kotlin class with no Android dependencies. It reuses `MetricsEngine` — the same computation engine used during live recording — to guarantee that calibration and runtime metrics are computed identically.

Prerequisites

- The track must be saved in **JSON format** (KML/GPX do not contain raw accelerometer data).
- The track must have been recorded with **Record Acceleration** enabled, so raw per-sample accelerometer arrays are present in each GPS fix.
- The track must contain the **gravity vector** captured at the start of recording (`meta.recordingSettings.calibration.baseGravityVector`).

Step 1 — Gravity Vector and Vehicle Basis

The stored gravity vector `[gx, gy, gz]` is read from the JSON track header:

```
"meta": {
  "recordingSettings": {
    "calibration": {
      "baseGravityVector": { "x": 0.12, "y": 9.76, "z": 1.03 }
    }
  }
}
```

`MetricsEngine.computeVehicleBasis()` is called once with this vector to produce the three orthonormal unit vectors (forward, lateral, vertical) that define the vehicle frame. All subsequent fix computations use this basis, ensuring the calibration analysis uses exactly the same coordinate frame as the original recording.

Step 2 — Sampling Rate Inference

The actual accelerometer sampling rate is inferred from the track data rather than assumed to be a fixed value. For each consecutive pair of GPS fixes that contain raw accelerometer samples:

```
rate_i = raw_sample_count_i / (gps_timestamp_i - gps_timestamp_{i-1}) [seconds]
```

The **median** of all per-interval rates is used as the effective sampling rate. The median is preferred over the mean to reject outliers caused by GPS timestamp jitter or brief recording gaps. This inferred rate is reported in the recommendation result for transparency.

Step 3 — Streaming Fix-by-Fix Analysis

The engine streams the JSON data array one GPS fix at a time to avoid loading the entire file into memory simultaneously. For each fix:

1. **Speed filter**: Fixes below `minSpeedKmph` are skipped. Low-speed data is unreliable for road quality classification because tyre noise and suspension behaviour differ significantly from normal driving.

2. **Raw sample extraction**: The `accel.raw` array for the fix is parsed into a `List<FloatArray>`. Each element is a three-axis accelerometer sample `[x, y, z]` in m/s^2.

3. **Metrics computation**: `MetricsEngine.computeAccelMetrics()` is called with the raw sample list, the GPS speed, and the vehicle basis. This function:

    - Removes gravity from each sample using the vehicle basis vertical unit vector
    - Projects linear acceleration onto forward, lateral, and vertical axes
    - Applies a moving-average detrend to remove low-frequency drift
    - Computes per-fix statistics: RMS, standard deviation, peak magnitude, mean vertical acceleration, forward/lateral max values
    - Maintains a rolling history of recent fix metrics for windowed averaging
4. **Result accumulation**: The returned `AccelMetrics` object is stored in a `FixResult` list. No raw samples are retained after this step.

Step 4 — Road Quality Threshold Recommendation

After all fixes are processed, the engine computes recommended thresholds from the empirical distributions of the accumulated fix metrics.

Smooth Threshold

The fix RMS values are sorted in ascending order. The smooth threshold is set at the **`smoothTargetPct`-th percentile** (default 60%):

```
sorted_rms = sort(fix_results.avgRms)
n = len(sorted_rms)
smooth_idx = floor(smoothTargetPct / 100 * n)
rms_smooth_max = sorted_rms[smooth_idx]
```

This means that with the recommended threshold, approximately `smoothTargetPct`% of the fixes from this track will be classified as smooth. The same percentile index is applied to the `avgStdDev` distribution to produce `std_dev_smooth_max`.

Rough Threshold

The rough threshold is set at the **`(100 - roughTargetPct)`-th percentile** (default 90th percentile, targeting 10% rough):

```
rough_idx = floor((100 - roughTargetPct) / 100 * n)
rms_rough_min = sorted_rms[rough_idx]
```

This ensures that approximately `roughTargetPct`% of fixes will be classified as rough. The same index is applied to `avgStdDev` for `std_dev_rough_min`.

A floor constraint is applied to prevent the rough threshold from being lower than the smooth threshold:

```
rms_rough_min = max(rms_rough_min, rms_smooth_max + 0.01)
std_dev_rough_min = max(std_dev_rough_min, std_dev_smooth_max + 0.01)
```

Feature (Bump/Pothole) Threshold

The `mag_max_severe_min` threshold is set so that the top `(bumpTarget + potholeTarget)` fixes by peak magnitude are classified as features:

```
sorted_mag_max = sort(fix_results.magMax)  // ascending
feature_target = bumpTarget + potholeTarget
mag_max_severe_min = sorted_mag_max[n - feature_target]
```

This is a direct percentile selection: if the user specifies 5 bumps + 5 potholes = 10 feature events, the threshold is set at the 10th-highest peak magnitude observed in the track.

Step 5 — Driver Threshold Recommendation

Hard Brake / Hard Acceleration Threshold

The forward-axis maximum (`fwdMax`) distribution is sorted. The threshold is set so that the top (`hardBrakeTarget + hardAccelTarget`) fixes are classified as hard events:

```
sorted_fwd_max = sort(fix_results.fwdMax)
brake_target = hardBrakeTarget + hardAccelTarget
fwd_threshold = sorted_fwd_max[n - brake_target]
hardBrakeFwdMax = hardAccelFwdMax = fwd_threshold
```

Swerve Threshold

The lateral-axis maximum (`latMax`) distribution is sorted. The threshold is set so that the top `swerveTarget` fixes are classified as swerves:

```
sorted_lat_max = sort(fix_results.latMax)
swerve_threshold = sorted_lat_max[n - swerveTarget]
swerveLatMax = swerve_threshold
aggressiveCornerLatMax = swerve_threshold * 0.8
```

The aggressive corner threshold is set at 80% of the swerve threshold to create a graduated severity band.

Step 6 — Achieved Percentage Verification

After computing all recommended thresholds, the engine makes a second pass over the fix results to compute the **achieved** smooth%, rough%, bump count, and pothole count under the recommended thresholds. These are reported alongside the recommendations so the user can verify the targets were met.

Calibration Parameters (User-Configurable)

Parameter	Default	Description
`smoothTargetPct`	60%	Percentage of fixes to classify as smooth
`roughTargetPct`	10%	Percentage of fixes to classify as rough
`bumpTarget`	5	Number of bump events to target
`potholeTarget`	5	Number of pothole events to target
`hardBrakeTarget`	5	Number of hard brake events to target
`hardAccelTarget`	5	Number of hard acceleration events to target
`swerveTarget`	5	Number of swerve events to target
`minSpeedKmph`	6 km/h	Minimum speed for fix inclusion

Applying Recommendations

The user can apply the recommended thresholds to: - **The current active profile** — updates the profile file on disk and writes all values to the live DataStore immediately, so they take effect without restarting. - **Any other named profile** — updates only the profile file; the live DataStore is not changed unless that profile is also the active one.

Implementation Classes

Class	Role
`ThresholdRecommendationEngine`	Orchestrates streaming parse, calls `MetricsEngine`, runs percentile searches
`MetricsEngine`	Shared computation engine — identical to runtime path
`CalibrateParams`	User-supplied calibration targets
`ThresholdRecommendation`	Result: recommended `CalibrationSettings` + `DriverThresholdSettings` + summary stats
`TrackHistoryViewModel`	Manages async calibration flow and UI state
`CalibrateParametersDialog`	Composable for user input of calibration targets
`ThresholdRecommendationDialog`	Composable showing current vs recommended values, apply buttons

---

KML Output and Visualisation

When KML output format is selected, Tracker generates a structured KML file with two types of content:

Road Quality Line Segments

The track is split into consecutive line segments, each coloured by road quality classification:

Quality	Colour
Smooth	Green
Average	Yellow
Rough	Red
Below Speed	Grey

Each segment groups consecutive fixes with the same road quality classification into a single `<LineString>` placemark, minimising file size while preserving colour transitions at every quality change.

Feature Placemarks

Detected bumps and potholes are written as individual `<Point>` placemarks with descriptive names and icons, placed at the GPS coordinate of the fix where the feature was detected.

Data Quality Considerations

GPS Accuracy

- Minimum satellite count for reliable data: 4 satellites
- Horizontal accuracy threshold: 20 metres
- Speed accuracy degrades at low speeds (< 5 km/h)

Accelerometer Limitations

- Device-specific sampling rates affect detection sensitivity
- Phone mounting orientation impacts measurement accuracy
- Temperature can affect sensor calibration

Filtering Strategies

- Points with GPS accuracy > 20 m are excluded from analysis
- Low-speed points (< `min_speed_kmph`) are excluded from driver metrics
- Moving-average detrending removes low-frequency drift from accelerometer data

---

Implementation Notes

Real-time Processing

- Metrics are computed in real-time during recording inside `TrackingService`
- `MetricsEngine` is a pure Kotlin class with zero Android dependencies, enabling reuse in both the recording path and the offline calibration engine
- Moving averages maintain a rolling window of recent fix metrics
- Event detection uses the same windowed history for both runtime and calibration

JSON Track Format

Each GPS fix in the JSON output contains: - `gps` object: timestamp, latitude, longitude, altitude, speed, course, accuracy - `accel.raw`: array of raw `[x, y, z]` accelerometer

samples captured during the GPS interval - `accel.metrics`: computed statistical summary (rms, stdDev, peakRatio, roadQuality, featureDetected, driver event) - `meta`: recording settings including the captured gravity vector

Memory Efficiency

- The calibration engine streams one fix at a time; only `FixResult` summaries (11 floats per fix) are retained in memory
- Raw accelerometer samples are parsed and discarded immediately after `computeAccelMetrics()` returns
- For a 1-hour track at 1 Hz GPS with 100 Hz accelerometer, peak memory for calibration is O(n) in the number of GPS fixes, not in the number of accelerometer samples

---

References

1. **Vehicle Dynamics**: Rajamani, R. "Vehicle Dynamics and Control"
2. **Road Roughness**: ISO 8608:2016 "Mechanical vibration — Road surface profiles"
3. **Accelerometer Theory**: IEEE Standard Specification Format Guide and Test Procedure for Linear Accelerometers
4. **GPS Accuracy**: Kaplan, E.D. "Understanding GPS/GNSS"

This documentation describes the current implementation as of version 1.1-Beta. Algorithms and thresholds may be refined in future versions based on field testing and user feedback.